Algorithm 5 Block-Sparse FlashAttention Forward Pass

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, on-chip SRAM of size M, softmax scaling constant $\tau \in \mathbb{R}$, masking function MASK, dropout probability p_{drop}, block sizes $B_c = \lceil \frac{M}{4d} \rceil, B_r = \min\left(\lceil \frac{M}{4d} \rceil, d\right)$, block sparsity mask $M \in \{0,1\}^{N/B_r \times N/B_c}$..

Initialize the pseudo-random number generator state $\mathcal{R}$ and save to HBM.
Initialize $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}, \ell = (0)_N \in \mathbb{R}^N, m = (-\infty)_N \in \mathbb{R}^N$ in HBM.
Divide $\mathbf{Q}$ into $T_r = \left\lceil \frac{N}{B_r} \right\rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ in to $T_c = \left\lceil \frac{N}{B_c} \right\rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_i, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_i, \dots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \dots, m_{T_r}$ of size B_r each.
for $1 \le j \le T_c$ **do**
  Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
  for $1 \le i \le T_r$ **do**
    if $M_{ij} \neq 0$ **then**
      Load $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
      On chip, compute $\mathbf{S}_{ij} = \tau \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
      On chip, compute $\mathbf{S}_{ij}^{\text{masked}} = \text{MASK}(\mathbf{S}_{ij})$.
      On chip, compute $\tilde{m}_{ij} = \text{rowmax}(\mathbf{S}_{ij}^{\text{masked}}) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij}^{\text{masked}} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\tilde{\ell}_{ij} = \text{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$.
      On chip, compute $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}$, $\ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\ell}_{ij} \in \mathbb{R}^{B_r}$.
      On chip, compute $\tilde{\mathbf{P}}_{ij}^{\text{dropped}} = \text{dropout}(\tilde{\mathbf{P}}_{ij}, p_{\text{drop}})$.
      Write $\mathbf{O}_i \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1}(\text{diag}(\ell_i) e^{m_i - m_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathbf{P}}_{ij}^{\text{dropped}} \mathbf{V}_j)$ to HBM.
      Write $\ell_i \leftarrow \ell_i^{\text{new}}$, $m_i \leftarrow m_i^{\text{new}}$ to HBM.
    end if
  end for
end for
Return $\mathbf{O}, \ell, m, \mathcal{R}$.

D Extension Details

D.1 Block-sparse FlashAttention

We describe the full block-sparse FlashAttention algorithm in Algorithm 5. The algorithm is identical to Algorithm 2, except that we skip zero blocks.

We prove the IO-complexity of block-sparse FlashAttention.

Proof of Proposition 4. The proof is very similar to the proof of Theorem 2. For the block-sparse case, notice that we only need to load blocks corresponding to nonzero blocks. As a result, the number of HBM accesses are scaled by s, the fraction of nonzero blocks in the block-sparsity mask. However, for small values of s, we would still need to write the result $\mathbf{O} \in \mathbb{R}^{N \times d}$. Therefore the number of HBM accesses is

$$\Theta\left(Nd + \frac{N^2 d^2}{M} s\right).$$

□

D.2 Potential Extensions

We discuss here a few potential extensions of the IO-aware approach to speed up deep learning training.

Multi-GPU Attention. Large language models are trained on hundreds or thousands of GPUs, and one typically splits the attention computation between 4-8 GPUs on the same node [77]. This introduces another level of memory hierarchy: beside GPU SRAM and GPU HBM, we also have the HBM of other

GPUs. For very long sequences, the different GPUs on the same node can cooperate to compute attention by taking into account the asymmetry of different levels of memory hierarchy.

Sparse MLP layers. Typical dense MLP layers are compute-bound and not memory-bound. To improve their efficiency, MLP layers with sparse weight matrices can be used [17]. However, many sparse MLP layers are instead memory-bound, and their speedup is often not proportional to the sparsity. We believe that an IO-aware implementation can alleviate this issue and realize the benefits of sparsity. We are excited about future work in this direction, to reduce the computational requirement of large models and improve their wall-block runtime.

Kernel machine learning. Our approach in FlashAttention relies on the fact that the $N \times N$ attention matrix is a function of a low-rank matrix $\mathbf{Q}\mathbf{K}^\top$ (of rank $d \ll N$). As a result, we can repeatedly load the inputs $\mathbf{Q}, \mathbf{K}$ and recompute the block of the attention matrix that we need, significantly reducing HBM access. As similar scenario happens in kernel machine learning: each element K_{ij} of the $N \times N$ kernel matrix $\mathbf{K}$ is a function of two vectors of size $d \ll N$, as it measures the similarity between two datapoints x_i and x_j. The KeOps library [8, 26] is a successful example of how reducing memory reads/writes can speed up kernel operations. We hope that this will motivate kernel methods that focus more on reducing IOs instead of just FLOPs.

E Full Experimental Results

E.1 BERT

We train BERT-large following the training procedure and hyperparameters of the reference MLPerf 1.1 implementation. In particular, we use the LAMB optimizer with learning rate 3.75e-3, with batch size 448, trained for at most 7100 steps. The training is stopped once the validation accuracy (for masked language modeling) reaches the target 72.0%, and the wall-clock run-time is measured. We train with FP16 precision using Apex AMP (with O2 optimization level).

We compare our results with the reported training speed from Nvidia that was submitted to MLPerf 1.1 (Table 1).

We use the same train / validation data split provided by MLPerf 1.1 reference implementation. In particular, we evaluate on the same 10000 validation examples as the baseline from Nvidia.

We train the model on 8×A100-80GB GPUs. Each training run takes between 16 and 19 minutes, and we average the results of 10 runs.

E.2 GPT-2

We use the standard implementations of GPT-2 [67] from Huggingface `transformers` library and from Nvidia's Megatron-LM repo. We follow the training recipe of the Megatron-LM repo.

We use an effective batch size of 512, and use gradient accumulation to fit into available GPU memory. We use the AdamW optimizer, with learning rate 6e-4 for GPT-2 small and 1.5e-4 for GPT-2 medium, and weight decay of 0.1. All models are trained with the same hyperparameters for 400K steps. We run all implementations with mixed-precision training (PyTorch AMP).

We use the Openwebtext dataset, with the GPT-2 BPE tokenizer. We randomly select 0.5% of the dataset as the validation set, with the rest being used as training set. This random selection of validation set is done once, and all models are evaluated on the same validation set.

We train the model on 8×A100-40GB GPUs, and we measure the wall-clock training time. Training GPT-2 small takes between 2.7-9.5 days, and training GPT-2 medium takes between 6.9-21.0 days (Table 2).

In Fig. 4, we plot of the validation perplexity throughout training of GPT-2 small/medium, using either HuggingFace implementation or our FlashAttention implementation. We see that FlashAttention behaves the same as the baseline implementation and the validation perplexity curves of the two implementations almost lie on top of each other.

Long Document Classification. For MIMIC-III and ECtHR, we follow the hyperparameters of Dai et al. [13].

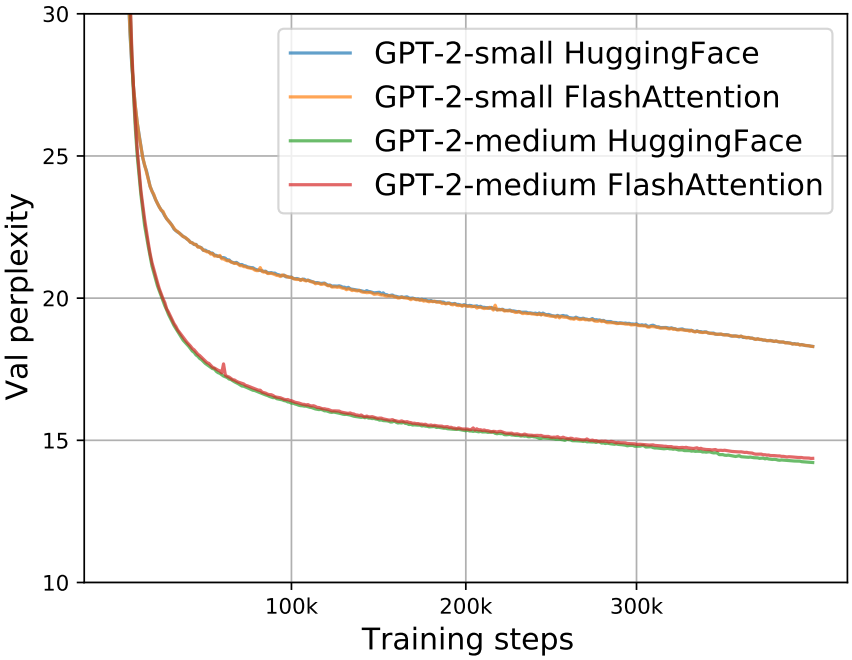

Figure 4: Validation perplexity of GPT-2 small/medium using two implementations. We confirm that FlashAttention yields the same validation curves as the baseline implementation from HuggingFace.

E.3 LRA details

We follow the hyperparameters from the Long-range arena paper [80], the Long-range arena repo (`https://github.com/google-research/long-range-arena`), and the Nyströmformer reproduction [90]. To be generous to the baseline methods, if we are unable to reproduce the performance of any baseline for any of the five tasks, we report the better performance from Tay et al. [80] or Xiong et al. [90] for that baseline on that task.

After hyperparameter tuning, almost all of the attention methods achieve similar accuracy on all of the five LRA tasks.

We run all methods with mixed-precision training, except for Performer (not stable with mixed precision) and Local Attention (implementation does not support FP16).

To calculate the overall wallclock-time speedup, we take the geometric mean of the wallclock-time speedup of each of the five tasks.

Path-X For Path-X and Path-256, we follow the hyperparameters from the PathFinder-32 experiments from the long-range arena paper[80]. For both, we first pretrain a model on Path-64. We take the checkpoint after 200 epochs, upsample its positional embedding (we duplicate the positional embeddings gridwise in space), and fine-tune it on the downstream task for 200 epochs with one epoch of linear warmup, and cosine decay of the learning rate. For Path-X, we take the best performing checkpoint (according to val accuracy), and additionally fine-tune it for 200 epochs with the same warmup and learning rate (this adds roughly 4 points of accuracy to FlashAttention for Path-X, but the model starts overfitting afterwards).

E.4 Comparison with Apex FMHA

We compare our method/implementation with Apex FMHA (`https://github.com/NVIDIA/apex/tree/master/apex/contrib/csrc/fmha`).

When we started this project, Apex FMHA was the fastest implementation of attention (that we knew of), tailored for short sequences of length at most 512. In fact, almost all MLPerf submissions for BERT training benchmark running on Nvidia GPUs use FMHA for their model code, as of MLPerf 1.1 [58]. Since